

DATA VISUALIZATION WEEKLY PROJECT

25MCA022

S.MEGAVARDHINI

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>3D Scatter Plot Projection</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1>3D Scatter Plot Projection</h1>

  <label for="projection">Projection:</label>
  <select id="projection">
    <option value="orthographic">Orthographic</option>
    <option value="perspective">Perspective</option>
  </select>
```

<label for="yaw">Yaw:</label>

<input type="range" id="yaw" min="-180" max="180"
value="0">

<label for="pitch">Pitch:</label>

<input type="range" id="pitch" min="-90" max="90"
value="0">

<label for="roll">Roll:</label>

<input type="range" id="roll" min="-180" max="180"
value="0">

<input type="checkbox" id="autoRotate" checked>

<label for="autoRotate">Auto Rotate</label>

<canvas id="canvas" width="900"
height="600"></canvas>

<script src="script.js"></script>

</body>

</html>

Script.js

```
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");
let points = [];
let yaw = 0;
let pitch = 0;
let roll = 0;
let projectionMode = "perspective";
const yawSlider = document.getElementById("yaw");
const pitchSlider = document.getElementById("pitch");
const rollSlider = document.getElementById("roll");
const projectionSelect =
document.getElementById("projectionSelect");
const autoRotate = document.getElementById("autoRotate");
fetch("data.json")
.then(res => res.json())
.then(data => {
    points = data;
    animate();
});
yawSlider.oninput = function() {
    yaw = yawSlider.value * Math.PI / 180;
```

```
};
```

```
pitchSlider.oninput = function() {  
    pitch = pitchSlider.value * Math.PI / 180;  
};
```

```
rollSlider.oninput = function() {  
    roll = rollSlider.value * Math.PI / 180;  
};  
projectionSelect.onchange = () => {  
    projectionMode = projectionSelect.value;  
};
```

```
function rotateX(p, a){  
    let c = Math.cos(a), s = Math.sin(a);  
    return {  
        x: p.x,  
        y: p.y*c - p.z*s,  
        z: p.y*s + p.z*c  
    };  
}
```

```
function rotateY(p, a){  
    let c = Math.cos(a), s = Math.sin(a);
```

```
return {  
    x: p.x*c + p.z*s,  
    y: p.y,  
    z: -p.x*s + p.z*c  
};  
}
```

```
function rotateZ(p, a){  
    let c = Math.cos(a), s = Math.sin(a);  
    return {  
        x: p.x*c - p.y*s,  
        y: p.x*s + p.y*c,  
        z: p.z  
    };  
}
```

```
function orthographic(p){  
    return {  
        x: p.x,  
        y: p.y  
    };  
}
```

```
function perspective(p){  
  const f = 1000;  
  const scale = f / (f + p.z);  
  
  return {  
    x: p.x * scale,  
    y: p.y * scale,  
    scale: scale  
  };  
}
```

```
function drawScene(){  
  
  ctx.fillStyle = "black";  
  ctx.fillRect(0,0,canvas.width,canvas.height);  
  
  const cx = canvas.width / 2;  
  const cy = canvas.height / 2;  
  
  const viewScale = 0.4;  
  
  for(let p of points){
```

```
let point = rotateX(p,pitch);
point = rotateY(point,yaw);
point = rotateZ(point,roll);

point.z += 250;

let screen;
let size = 1;

if(projectionMode == "orthographic"){

    screen = orthographic(point);

    let x = cx + screen.x * viewScale;
    let y = cy - screen.y * viewScale;

    drawPoint(x, y, size);

} else {

    screen = perspective(point);

    size = Math.max(1.5, 6 * screen.scale);
```

```
    let x = cx + screen.x * viewScale;
    let y = cy - screen.y * viewScale;

    drawPoint(x, y, size);
  }
}
```

```
function drawPoint(x, y, size){

  ctx.beginPath();
  ctx.arc(x, y, size, 0, Math.PI * 2);

  ctx.fillStyle = "#7C4DFF";
  ctx.shadowColor = ctx.fillStyle;
  ctx.shadowBlur = 2;

  ctx.fill();
  ctx.shadowBlur = 0;
}
```



```
function animate(){  
  
    if(autoRotate.checked){  
  
        yaw += 0.02;  
        pitch += 0.01;  
        roll += 0.01;  
  
        yawSlider.value = (yaw*180/Math.PI)%360;  
        pitchSlider.value = (pitch*180/Math.PI)%360;  
        rollSlider.value = (roll*180/Math.PI)%360;  
    }  
  
    drawScene();  
  
    requestAnimationFrame(animate);  
}
```

Project Structure:

